

Warm up

What is the value of `result` after executing the following snippet?

```
int x = 8;  
int* xp = &x;  
int result = *xp * !!xp;
```

Warm up

What is the value of `result` after executing the following snippet?

```
int x = 8;  
int* xp = &x;  
int result = *xp * !!xp;
```

Answer: `result = 8`

- `xp` is a pointer to `x`, so `*xp` is the value of `x`, which is 8
- `!!xp` checks if `xp` is non-null, which is true, so `!!xp` evaluates to 1
- Therefore, `result` is $8 * 1 = 8$

Precept 6: The Power of Pointers (and Arrays)

Polly Ren

COS 217: Introduction to Programming Systems
Spring 2026
Princeton University

February 12, 2026

Overview

1 Pointers

2 Arrays

3 Summary

What is a pointer?

- A pointer is an integral variable that stores a memory address
- Declared using the `*` operator
 - `int *p;` declares a pointer `p` that can point to an `int`
 - `char *s;` declares a pointer `s` that can point to a `char`
 - `int **pp;` declares a pointer `pp` that can point to a pointer to an `int`
 - `int *a, *b;` declares pointers `a` and `b` that can each point to an `int`
 - `int *a, b;` declares a pointer `a` that can point to an `int` and an `int` variable `b`
- For a value `v`, `&v` says “what is the memory address of `v`”
 - Can only apply `&` to l-values (variables that can be assigned to), e.g., `&x` is valid but `&5` is not
- For a pointer `p`, `*p` says “visit the address stored in `p` and get the value there”
 - Can only apply `*` to pointer variables, e.g., `*p` is valid but `*5` is not
- `&` and `*` are inverses of each other, e.g., `*&x` is `x` and `&*p` is `p`

Pointer examples

See `ptrs.c` for full example

```
int i1, i2;      /* i1, i2 are variables of type int. */
int *pi3, *pi4; /* pi3, pi4 are variables of type int* (int pointer). */
i1 = 5;          /* Assign the value 5 to variable i1. */
pi3 = &i1;       /* "&" is the "address of" operator. pi3 points to i1. */
*pi3 = 6;        /* "*" is the "dereference" operator. */
                /* Assign the value 6 to location pi3 points to, i1. */
pi4 = &i2;       /* Hereafter, *pi4 and i2 are aliases. */
i2 = *pi3;      /* Assign the value of *pi3 (i1) to i2. */
*pi4 = *pi3;    /* Assign the value of *pi3 (i1) to *pi4 (i2). */
pi4 = pi3;      /* Assigns address pi3 to pointer pi4. */
                /* pi3 and pi4 are now aliases. */
pi4 = &i2;      /* Restore pi4 to previous value */
pi3 = NULL;     /* Indicates that pi3 points to address zero. */
```

Comparing pointers

- Pointers are just numbers $\rightarrow$ we can compare them using relational operators
- Pointer comparison is only really meaningful if both pointers point to elements of the same array
- Check if two pointers point to the same location: `p1 == p2`
 - If `p1 == p2`, then `*p1` and `*p2` refer to the same spot in memory, so `*p1 == *p2` is usually true
 - ◇ Caveat: if `p1` and `p2` point to the same location but have different types, then `*p1 == *p2` may not be true

```
int x = 10, y = -1, z = -1;
int *px = &y;
long long *py = (long long *)&y;
/* px = 0x16d9267a4, py = 0x16d9267a4 */
/* *px = -1, *py: 47244640255 */
```
 - Takeaway: `p1 == p2` does not necessarily mean `p1` and `p2` are the same type

Why are pointers useful?

- Arrays (in a bit)
- Dynamic memory allocation (next week)
- Call by reference: pass a pointer to a function so that the function can modify the caller's variable
 - Sometimes it is expensive to make copies of large data structures to pass to a function
 - Use out-parameters to “return” multiple values from a function
 - ◇ `scanf("%d", &i);` allows `scanf` to modify the caller's variable `i`

Swap two numbers: a first attempt (incorrect)

```
static void swap(int iFirst, int iSecond) {  
    int iTemp;  
    iTemp = iFirst;  
    iFirst = iSecond;  
    iSecond = iTemp;  
}  
  
int main(void) {  
    int i1 = 17;  
    int i2 = 38;  
    swap(i1, i2);  
    return 0;  
}
```

Why does this not work?

- Because swap is modifying its local copies of iFirst and iSecond, not the caller's variables i1 and i2

Swap two numbers (correct)

```
static void swap(int *piFirst, int *piSecond) {  
    int iTemp;  
    iTemp = *piFirst;  
    *piFirst = *piSecond;  
    *piSecond = iTemp;  
}  
  
int main(void) {  
    int i1 = 17;  
    int i2 = 38;  
    swap(&i1, &i2);  
    return 0;  
}
```

Why does this work?

- We are passing the caller's copy of `i1` and `i2` to `swap` by reference, so `swap` can directly modify the caller's variables

Overview

1 Pointers

2 Arrays

3 Summary

What is an array?

- Arrays are:
 - Homogeneous: all elements have the same type
 - Contiguous: all elements are stored in adjacent memory locations
 - ◇ Elements are accessed using an index, numbered 0 to $n-1$ for an array of size n
 - ◇ `arr[i]` is stored immediately before `arr[i+1]` in memory
 - Sequential: element 0 has the lowest address, element 1 has the next-lowest address, etc.
 - ◇ Index i of an array of type `type` is stored at an offset of $i * \text{sizeof}(\text{type})$ bytes from the start of the array
 - Fixed-size: the size of an array is determined at compile time and cannot be changed at runtime (for non-heap-allocated arrays)
- Can be stored on the stack, heap or in the data segment

Array syntax

- Declare an array: `int arr1[5];`
- Declare an array with initial values: `int arr2[] = {1, 2, 3};`
 - The size of `arr2` is determined by the number of initial values, so `arr2` has three elements
 - Curly braces can only be used to initialise a newly-declared array
- Access an array element: `int x = arr1[3];`
 - Retrieve element 3 of `arr1` and assign it to `x`
- Update an array element: `arr2[1] = 42;`
 - Update element 1 of `arr2` to be 42
- What does `arr2[3] = 99;` do?
 - It attempts to update element 3 of `arr2` to be 99
 - But, we've only allocated enough space for three elements in `arr2`
 - C does not perform bounds checking!!
 - Element 3 is out of bounds → undefined behaviour
 - ◇ May corrupt adjacent memory, cause a segmentation fault or even appear to work correctly (out of pure luck)

News flash: Arrays are really pointers

- In C, an array name is a pointer to the first element of the array
- `arr2` is a pointer to the first element of `arr2`,
 - `arr2` and `&arr2[0]` are equivalent
- `arr2[i]` is syntactic sugar for `*(arr2 + i)`
 - Pointer arithmetic!

Pointer arithmetic

- For an array name `arr`, `arr + i` is the address of the element that is i elements after the first element of the array
 - To read or write to element at index i , use `arr[i]` or `*(arr + i)`
 - Oddly enough, `i[arr]` is also valid and equivalent (!)
 - ◇ `arr[i] = *(arr + i) = *(i + arr) = i[arr]`
- Suppose we have an array `arr` of four ints
 - Suppose the array is allocated starting at address 100, i.e., `arr = 100`
 - An int is 4 bytes $\rightarrow$ the array is $4 \times 4 = 16$ bytes in size
 - `arr + 0 = 100` $\rightarrow$ `arr[0]` is stored at address 100
 - `arr + 1 = 104` $\rightarrow$ `arr[1]` is stored at address 104
 - `arr + 2 = 108` $\rightarrow$ `arr[2]` is stored at address 108
 - `arr + 3 = 112` $\rightarrow$ `arr[3]` is stored at address 112
 - `arr + i = 100 + 4i` $\rightarrow$ `arr[i]` is stored at address $100 + 4i$
- Extends to pointers: For a pointer `p` of type `type*`, `p + i` is the address of the element i elements after the one pointed to by `p`
 - The offset is calculated as `i * sizeof(type)` bytes

Overview

1 Pointers

2 Arrays

3 Summary

Summary

- Pointers: declaration, dereferencing, call by reference, out-parameters
- Arrays: what they are, declaration, pointer arithmetic
- Dates to remember:
 - Assignment 2: due Tue, 2/24 at 9pm